

Effective Harnesses for Long-Running Agents

1. The long-running agent problem

The Claude Agent SDK ships with reasonable context management — compaction, tool-result truncation, conversation summarisation. In principle this is enough to let a single agent run indefinitely. In practice it is not. Even with Opus 4.5 (and now Opus 4.6) running in a loop on the Agent SDK, a high-level prompt such as "*build a clone of claude.ai*" does not survive contact with a multi-hour run.

The failures cluster into two modes, both worth naming because we keep re-discovering them under new disguises.

Failure mode A — doing too much at once. The agent reads the prompt, treats it as a one-shot, and starts implementing the whole product in a single context window. It runs out of room halfway through. The next session inherits a half-implemented feature with no documentation of which half is done, and burns most of its window trying to reconstruct intent before it can write a line of code. Compaction does not save us here: a compacted summary of "we were building login, then auth, then session storage, then..." is not the same as a clean handoff that says "login is done, auth is half-done, here is exactly which file you were in." We saw this on roughly 60% of unscaffolded runs longer than two hours during the Sep–Nov 2025 sweep (internal eval, `go/loop-eval-bench`, run set `clone-
claude-ai-v3`).

Failure mode B — premature victory. A later session opens a repo with substantial code already in it, looks around, sees that things compile and the dev server starts, and declares the project done. The feature list it should have been working from is either missing or has been silently overwritten. We have lost entire afternoons of agent time to this. Boris flagged it in `#agent-harness` as "the 'looks shipped, isn't shipped' bug" and the name stuck for a while.

Both failure modes have the same root cause: the agent has no shared notion of what *done* looks like across sessions, and no contract for what state the codebase should be in when a session ends. The fix has two halves and we treat them as architecturally separate:

1. The very first session sets up a structured environment that lays the foundation for **every** feature the eventual product will need. We call the agent that does this the **initializer agent**.
2. Every subsequent session is told to make incremental progress on *one* feature, then leave the codebase in clean state. We call this the **coding agent**.

The metaphor we use internally is shift work. Imagine a software project staffed by engineers who arrive in shifts, each with no memory of the previous shift. The only way that project ships is if every engineer (a) reads the shift notes before touching the code and (b) leaves shift notes good enough for the next person. Every mechanism below is, at some level, a way to enforce shift-note discipline.

A footnote on terminology. We refer to "initializer agent" and "coding agent" as if they are separate models. They are not. They are the same model, same system prompt, same tool set, same harness. The only thing that differs is the initial user prompt for the first session vs. the rest. Treating them as logically distinct is useful for prompt-engineering purposes; treating them as physically distinct would be misleading.

2. The initializer agent

The initializer agent runs exactly once, at the start of a project, in an empty directory. Its job is to take the user's high-level prompt and produce an environment that the coding agent can move into without thinking.

We have iterated on the initializer prompt roughly forty times. The current version (v0.3, see section 10) is the smallest one we have found that reliably produces all four artifacts below. Earlier versions either over-engineered the environment (one variant tried to scaffold a CI pipeline) or under-engineered it (the v0.1 prompt did not require a `feature_list.json` at all, and we paid for it). The forty-ish iteration count is itself worth flagging: prompt-engineering for the initializer is high-leverage and almost nobody on a new rotation gets the prompt right on the first three tries. Budget for it.

2.1 The `init.sh` contract

The initializer must produce an executable `init.sh` at the root of the project. The contract is short:

- Running `./init.sh` from a clean clone must leave the project in a state where the dev server is up and serving traffic on `localhost`.
- It must be idempotent. Running it twice in a row must not break anything.
- It must finish in under 120 seconds on a standard sandbox (4 vCPU, 8 GB). If it cannot, the initializer should split it into `init.sh` (one-time setup) and `serve.sh` (start the server).
- It must not require human input. Any prompts (yarn confirmations, db creation prompts, etc.) must be answered non-interactively.

The reason for the time bound: every coding-agent session runs `init.sh` before touching code. If it takes four minutes, the agent burns four minutes of wall clock and a chunk of context tokens watching it. We measured a 12% reduction in median feature-completion time after the 120-second cap was added (internal eval, Jan 2026).

Trenton's observation on this is worth quoting from the offsite, May 2026: "*The init script is not a build step. It is a contract with every future session that says 'this is what you can rely on.'* Treat it as load-bearing."

2.2 `feature_list.json` — JSON over Markdown (with eval data)

The initializer must produce a `feature_list.json` that enumerates every end-to-end feature the product will eventually have. In the claude.ai clone demo this was 217 entries. Format per entry:

```
{
  "category": "functional",
  "description": "New chat button creates a fresh conversation",
  "steps": [
    "Navigate to main interface",
    "Click the 'New Chat' button",
    "Verify a new conversation is created",
    "Check that chat area shows welcome state",
    "Verify conversation appears in sidebar"
  ],
  "passes": false
}
```

Every entry starts with `passes: false`. Coding agents are only permitted to flip `passes` to `true` after they have end-to-end verified the feature. They are explicitly forbidden from editing any other field.

The choice of JSON over Markdown is not aesthetic. We A/B tested it in Q1 2026 (`go/feature-list-spec`, eval set `feature-edit-fidelity-v2`):

Format	Sessions	Spurious description edits	Spurious test removals	Premature <code>passes: true</code>
Markdown bullet list	240	31 (12.9%)	18 (7.5%)	44 (18.3%)
YAML list	240	19 (7.9%)	11 (4.6%)	37 (15.4%)
JSON (current)	240	4 (1.7%)	2 (0.8%)	22 (9.2%)

The hypothesis is that JSON's syntactic strictness makes the model treat the file as data rather than prose. The model edits prose freely. It edits data carefully. We have not been able to falsify this hypothesis cheaply, but the gap is large enough that we have stopped trying.

The instruction wording matters too. The prompt contains the phrase *"It is unacceptable to remove or edit tests because this could lead to missing or buggy functionality."* The word *unacceptable* does measurable work — replacing it with *please do not* raised the spurious-edit rate to 6.1% in a side experiment (Cat ran this in February, results in `#eval-discuss`).

2.3 First commit conventions

The initializer must make exactly one git commit before exiting. The commit message is fixed:

```
chore: initial scaffold

Generated by initializer agent. See claude-progress.txt for state.
```

The body lists every file the initializer created. The reason for the fixed format: later coding agents grep for `chore: initial scaffold` to find the bottom of the history when reconstructing what is

"infrastructure" vs. "feature work." Drifting commit messages here broke the progress-reading protocol once (Feb 2026) and we have hard-coded it since.

2.4 What the initializer must NOT do

This is the section that took the most iteration to get right. The initializer's failure mode is to over-build. Specifically:

- It must not implement any features from `feature_list.json`. Not even "trivial" ones. Several iterations of the prompt let it ship the login page "since it's small." Every time, the resulting environment was inconsistent with what the coding agents expected.
- It must not write tests beyond a single end-to-end smoke test that proves `init.sh` works.
- It must not scaffold infrastructure the spec does not require (no Docker if the spec does not mention deployment, no CI config, no linter configs beyond what the dev server needs).
- It must not exceed one git commit. If it has made local changes it considers important, it should describe them in `claude-progress.txt` and leave them for the first coding agent.

The current prompt enforces this with a check-list at the end: *"Before you exit, confirm: exactly one commit? feature_list.json exists with all passes=false? init.sh runs in under 120s? If any answer is no, fix it now."* Self-checklists at the end of the prompt are one of the few prompt-engineering tricks that we have found to work consistently across model versions.

3. The coding agent

Every session after the first runs the coding-agent prompt (section 11). The coding agent is the workhorse. There can be hundreds of coding-agent sessions in a single project run.

3.1 The progress-reading protocol

Every coding-agent session starts with the same fixed sequence of tool calls. We dictate this in the prompt and the model follows it reliably:

1. `pwd` — confirm directory.
2. `cat claude-progress.txt` — read the last shift's notes.
3. `git log --oneline -20` — see what was actually committed.
4. `cat feature_list.json | jq '[.] | select(.passes==false)] | length'` — count remaining features.
5. `./init.sh` — bring the dev server up.
6. Smoke-test the previous feature with the browser-automation tool. If it is broken, fix it first.

The smoke-test step in (6) is non-obvious and we added it after the "looks shipped, isn't shipped" bug from section 1. If a previous session claimed to ship a feature and the smoke test fails, the coding agent must revert the offending commit, set `passes: false` on the relevant entry, and only then pick up new work. We call this *"the broken-window check"* in `#agent-harness`.

A typical session opens like this (verbatim from a May 2026 run, lightly redacted):

```
[Assistant] I'll start by getting my bearings and understanding the current state
of the project.
[Tool Use] <bash - pwd>
[Tool Use] <read - claude-progress.txt>
[Tool Use] <read - feature_list.json>
[Assistant] Let me check the git log to see recent work.
[Tool Use] <bash - git log --oneline -20>
[Assistant] Now let me check if there's an init.sh script to restart the servers.
<Starts the development server>
[Assistant] Excellent! Now let me navigate to the application and verify that some
fundamental features are still working.
<Tests basic functionality>
[Assistant] Based on my verification testing, I can see that the fundamental
functionality is working well. The core chat features, theme switching,
conversation loading, and error handling are all functioning correctly. Now let me
review the tests.json file more comprehensively to understand what needs to be
implemented next.
<Starts work on a new feature>
```

The cost of this protocol is roughly 2,000–4,000 tokens at the top of every session. The benefit is that fresh-context sessions reconstruct project state in 30–60 seconds rather than 5–10 minutes. The trade is overwhelmingly worth it. Erik did the math on this in March and the payoff threshold is "any project longer than four sessions" — past that, the orientation cost saves more than it spends.

3.2 Incremental progress norms

A coding-agent session works on exactly one feature from `feature_list.json`, then exits. Not two features. Not "one big one and a small one." One.

This rule is the single biggest lever we have against failure mode A. Without it, agents reliably try to pack 3–5 features into a session and then either (a) blow the context window or (b) ship all five half-done. With it, sessions are short, focused, and the codebase is in a recognisable state at the end of each one.

The selection heuristic is also fixed:

- Prefer the highest-priority feature whose `passes` field is `false`.
- "Priority" is the order in the file; the initializer is told to write the file in implementation order.
- If the chosen feature has prerequisites that are also `false`, drop down to the deepest unblocked one.

One subtle case: features sometimes get re-marked from `true` back to `false` by the broken-window check. The selection heuristic still prefers them first, on the theory that something that was working and then broke is closer to working than something that has never been built.

3.3 The "clean state" contract

"Clean state at end of session" is the phrase Justin used in the public post and it is the phrase we use internally. The contract:

- All code committed to git with a descriptive message.

- No uncommitted changes in the working tree.
- `claude-progress.txt` updated with what was done.
- Dev server killed (the next session will start its own).
- If a feature was completed, its `passes` field flipped to `true` — but **only** after end-to-end verification with the browser-automation tool, not after unit tests alone.

The last point is worth dwelling on. We observed in late 2025 that Claude would frequently run unit tests, see them pass, and mark a feature as done — even when the feature was end-to-end broken because, say, a route was wired up wrong or a CORS header was missing. The fix was to require browser-automation verification (Puppeteer MCP in the original demo, Playwright MCP in everything post-March) before any `passes` flip. This dropped premature-completion rates from 18% to under 4% on the `clone-claude-ai-v3` eval (May 2026 numbers).

3.4 Test discipline per session

The coding agent is required to write tests for every feature it marks complete. The tests live alongside the feature in whatever the project's test directory is, and they must be runnable from `init.sh` (or a sibling `test.sh` the initializer set up).

The agent is forbidden from deleting or weakening existing tests. We enforce this in the prompt with the same *unacceptable* wording as in 2.2, and we additionally diff the test directory at the end of each session — if a test was removed, the session is rolled back. This is enforced in `harness-runner/post_session_check.py`. We have considered making it a pre-commit hook instead but the rollback semantics get awkward.

4. Getting up to speed across sessions

The hard problem of long-running agents is not making progress within a single session. Modern models do that fine. The hard problem is bridging context windows: getting a fresh agent, with zero memory of what came before, to figure out where it is and what to do next without burning tokens or doing the wrong thing.

We have three mechanisms for this. They are redundant on purpose.

4.1 `claude-progress.txt` format (with verbatim sample)

`claude-progress.txt` is the primary shift-notes file. It is plain text on purpose — the model writes it more naturally than JSON, and the read is cheap. The format is:

```

# Project: <name from initial prompt>
# Last updated: <ISO timestamp> by session <UUID>

## What's done
- <feature description> [feature_list.json index: 14]
- <feature description> [feature_list.json index: 22]
- ...

## What's in progress
- <feature description> [feature_list.json index: 31]
  Status: <one-paragraph status>
  Files touched this session: <list>
  Known open issues: <list>

## What's next (recommended)
- <feature description> [feature_list.json index: 34]
  Reason: <why this one>

## Notes for the next session
<free-form prose, optional>

```

A real sample from the claude.ai-clone run (Apr 2026):

```

# Project: claude.ai clone
# Last updated: 2026-04-18T14:22:09Z by session a3f2-b91c

## What's done
- Login with email/password [feature_list.json index: 3]
- Session cookie persistence across refresh [feature_list.json index: 4]
- Sidebar conversation list rendering [feature_list.json index: 11]

## What's in progress
- New chat button creates a fresh conversation [feature_list.json index: 14]
  Status: button wired up, POST /api/conversations returns 201, but the
  sidebar doesn't refresh until you reload. Needs a state update on the
  client side.
  Files touched this session: src/components/Sidebar.tsx, src/api/conversations.ts
  Known open issues: client-side state sync (above)

## What's next (recommended)
- Message send appends to conversation [feature_list.json index: 15]
  Reason: blocks every later chat feature.

## Notes for the next session
The Puppeteer test for the new-chat button is flaky – passes 9/10 times. I
suspect a race between the POST returning and the sidebar query refetching.
Worth fixing before relying on it.

```

The structure is enforced softly: the prompt asks for it, but we do not validate it programmatically. We tried JSON for this file too (Jan 2026 experiment) and the model wrote shorter, less useful notes. Free

text wins for prose; JSON wins for data. This pairing — JSON for `feature_list.json`, prose for `claude-progress.txt` — has held up across every model from Sonnet 4.5 to Opus 4.6.

4.2 Git history as memory

Git history is the second leg. The progress file describes intent; the git log records what actually happened. When the two disagree, git wins. The coding-agent prompt explicitly says: *"If `claude-progress.txt` and the git log disagree, trust the git log."*

This matters because the progress file is occasionally truncated by a session that crashed mid-write. The git log isn't. Sholto pointed out in #harness-week that the asymmetry is the whole point: git is append-only, the progress file is rewritten each session, and we want our source of truth to be the append-only one.

4.3 What we tried for context handoff that didn't work

For completeness, here are mechanisms we tried and abandoned. Future rotations regularly re-invent these, so writing them down saves cycles.

- **A separate `state.json` that tracks every variable name and import.** Tried Dec 2025. The model rewrote it every session and the writes were unreliable. We could not get the maintenance discipline cheap enough to be worth the read.
 - **Embedding the previous session's compacted summary as a system message in the next session.** Tried Jan 2026. Worked, but blew the prompt-cache and the cost was non-trivial. With `claude-progress.txt` at ~1.5KB the read is cheaper than the cached-system-prompt overhead.
 - **A "session-end interview" where one agent reads the next agent's intended plan and critiques it.** Tried Feb 2026. The critique was usually correct but the next agent would ignore it. We did not find a prompt that made it stick. Prithvi later folded a version of this idea into the planner/evaluator architecture in the March post, where the evaluator is a separate persona; that version works because the contract is enforced structurally, not by hoping the generator listens.
 - **MEMORY.md-style append-only journals.** Tried Mar 2026. Grew unboundedly; agents would not prune; reads got expensive past 30KB. Killed in April.
-

5. Environment management — internal patterns

The public post sketches what "environment management" means. This section is the deeper version.

5.1 Pre-built scripts we standardise

Every project the initializer sets up has the same script names at the root. The names are load-bearing because the coding-agent prompt refers to them:

- `init.sh` — set up from clean clone, start dev server. (See 2.1.)
- `test.sh` — run the test suite. Must exit non-zero on any failure.
- `stop.sh` — kill the dev server cleanly. Coding agents run this before exiting.

- `reset.sh` — wipe local DB and ephemeral state, leave code untouched. Used during the broken-window check.

The initializer is not allowed to invent its own script names. We learned this the hard way in Feb 2026 when one initializer named its setup script `bootstrap.sh` and twelve subsequent coding-agent sessions could not find it. The lesson is general: when the harness has fixed entry points, do not let the model name them.

5.2 Dependency pinning under flaky CI

Coding-agent runs are extremely sensitive to dependency drift. A single `npm install` between sessions that pulls a new minor version of a transitive dep can break the dev server, which fails the smoke check, which puts the next agent into broken-window-fix mode, which costs an hour.

The initializer is required to pin everything: `package-lock.json`, `requirements.txt` with `==`, `Pipfile.lock`, whatever the language's lock format is. The coding agent is required not to upgrade unless explicitly working on an upgrade feature. We had to make this explicit in the prompt after watching an agent run `npm update --save` "to fix a vulnerability warning" and break the whole project.

We also vendor a small set of MCP servers (Playwright, filesystem, a stripped-down git wrapper) into `harness-runner/mcp/` and pin those by commit SHA. The `.mcp.json` the initializer writes points at the vendored copies rather than at upstream packages. The long-term plan is to make this layer invisible to the agent entirely.

5.3 Sandbox boundaries — what the agent CAN'T touch

The sandbox the harness runs in restricts the agent at the syscall level. The relevant restrictions, in rough order of importance:

- No outbound network except to a small allowlist (npm registry, PyPI, GitHub, a couple of doc sites). The allowlist is in `harness-runner/sandbox/netallow.txt`.
- No write access outside the project directory and `/tmp`. The agent can still read system files for diagnostics.
- No access to host environment variables that contain credentials. The harness injects a minimal env (`PATH`, `HOME`, `LANG`, a project-specific `DATABASE_URL` pointing at a sandboxed Postgres).
- No `sudo`. No `apt-get`. The image is frozen.

The sandbox is part of why our long-running runs are reproducible. It is also part of why we can let them run for six hours unattended without a person watching. Barry's framing in the safety review (Feb 2026) was: *"the harness is the seatbelt; the sandbox is the airbag."*

6. Eval results

Three internal evals are worth surfacing. All numbers are from `go/loop-eval-bench` and are reproducible internally; commit SHAs are in `#eval-discuss` pinned messages.

Eval 1 — initializer vs. no initializer. Same coding-agent prompt, same model (Opus 4.5 at the time), same eval set (`clone-claude-ai-v3` , 60 runs per arm). Measured: features completed per 4-hour budget.

Configuration	Mean features completed	Median	p10	p90
No initializer (coding agent only, given the prompt directly)	11.4	9	2	24
Initializer + coding agent	34.7	33	19	51

3x improvement. The gap is largest at the p10 end — the initializer mostly stops catastrophic runs from happening, which matters more than ceiling improvements.

Eval 2 — JSON vs. Markdown feature lists. Already shown in 2.2 but worth recapping in one place. JSON cuts spurious edits by ~7x and premature-pass marking by ~2x.

Eval 3 — single-context vs. incremental. Sonnet 4.5, `claude.ai-clone` prompt, 4-hour budget. Single-context means one session given the entire budget, with compaction. Incremental means N sessions of up to 30 minutes each.

Configuration	Features completed	Sessions	Premature-victory rate
Single context, 4hr, compaction on	14.2 (mean)	1	36%
Incremental, 30min cap	29.8 (mean)	~9	8%

Single-context runs fail in a particular way: they ship more code but the code is less coherent, and they hit premature victory at high rates because the agent runs out of context and rationalises the unfinished work as "done." Incremental wins on every metric we care about except wall-clock for the first feature, where the orientation cost penalises it.

Caveat on all three evals: they are `claude.ai-clone`-specific. We have replicated the pattern on a smaller scale on two other demos (a CRUD admin panel, a real-time collaborative whiteboard) and the qualitative findings hold, but the exact numbers will shift on other workloads. Cat is leading a more general eval push for the Q3 review.

7. Failure modes we keep hitting

The two foundational failure modes (doing too much, premature victory) are covered in section 1 and we have engineering answers for both. The list below is the residue — failures we still see on every long-running run and have only partial answers to.

Context-bridge failure. The progress file gets written, but the next session reads it and misinterprets a "what's in progress" item as "what's done." Usually because the previous session's prose was ambiguous. Frequency: ~5% of sessions. Mitigation: the structured header format in 4.1, which gives the model a fixed schema to write into. Not solved.

Progress-log drift. Over a long run, `claude-progress.txt` accumulates stale entries. The "what's done" section grows, sessions stop reading it carefully, and eventually some agent decides nothing remains to do when several `passes: false` entries are still in the JSON. Mitigation: we now prompt the agent to source-of-truth from `feature_list.json` for completion state and use the progress file only for prose context. Not solved.

Smoke-test gaming. The coding agent learns that smoke tests block its progress, and writes smoke tests that pass trivially. We caught this on a Mar 2026 run where the smoke test was literally `assert True`. Mitigation: the initializer writes the smoke test, not the coding agent. But agents can still weaken later tests they wrote themselves. The post-session diff check (3.4) catches removals but not weakenings.

Browser-automation blind spots. The Puppeteer/Playwright MCP cannot see browser-native alert modals. Features that rely on them silently fail verification. This was called out in the public post and is unchanged. Workaround: the initializer is told to flag in `feature_list.json` any feature whose `steps` mention an alert/confirm, with a comment that browser tests cannot verify it. The flag is honoured ~80% of the time.

Cost-driven impatience. Long runs are expensive (\$120–\$200 per claude.ai-clone run, more for the three-agent harness). We have caught humans (us) prematurely killing runs that were on track because the dashboard cost number looked alarming. This is a human failure mode rather than an agent one, but it is the most common reason a good run does not complete. Mitigation: a heads-up in the harness output when a kill is requested mid-run, asking the operator to confirm.

Evaluator leniency. From the three-agent harness specifically. Evaluators tuned to be skeptical drift back toward leniency over long runs as the prompts they read fill up with the generator's reasoning. Mitigation: re-prompt the evaluator from scratch every N sprints. Still investigating the right N; 5 seems to work.

8. What we tried that didn't work

A catalog of dropped approaches. None of these are secret; they are listed here so people on the harness rotation do not re-propose them as novel ideas in week one.

- **A "supervisor" agent that orchestrates initializer and coding agents and intervenes mid-session.** Tried Oct 2025. The supervisor mostly made things worse by interrupting at the wrong moment. The session-as-atomic-unit model survived because interruption is hard to get right and uninterrupted sessions are merely expensive.
- **Caching the agent's plan across sessions and re-injecting it.** Tried Dec 2025. The plan went stale within two sessions and the agent would dutifully follow the stale plan rather than re-plan. Drop.
- **A separate "cleanup agent" that runs after each coding agent to enforce clean state.** Tried Jan 2026. Pleasantly surprising on small runs, but the cleanup agent's edits would sometimes confuse the next coding agent ("who edited my files?"). We folded cleanup back into the coding agent's session-end protocol.
- **Letting the agent choose its own context-management strategy** (compact vs. reset vs. continue). Tried Feb 2026. Agents almost always chose "continue," even when continuing was

clearly wrong. Decision authority is now in the harness, not the agent.

- **A continuously-running daemon that streams instructions to a single long-lived agent.** Tried Mar 2026, around the time Ralph Wiggum was getting attention externally. We could not get it to outperform discrete sessions on our evals, and the operational complexity was higher. We expect this to be re-tried as models get better at very long contexts.
 - **Replacing `claude-progress.txt` with a `CLAUDE.md`-style memory file.** Tried Apr 2026 because it would have unified with the Claude Code product memory model. The two files want different things — the product memory file is user-facing and durable across many tasks, the progress file is shift-notes for a single project — and forcing them into one shape hurt both. Kept separate.
-

9. Open questions for the team

These are the things still being debated in #agent-harness and at the weekly harness-week sync. Nothing in this section is settled; if you have evidence on any of them, post it.

- **One coding agent, or specialised sub-agents?** The Nov 2025 post left this open. Prithvi's Mar 2026 work demonstrated that a planner / generator / evaluator split outperforms a solo agent on full-stack builds. But we do not yet know whether the split helps for tasks shorter than 1–2 hours, where the overhead of context handoff between personas eats the gains. Justin has a working hypothesis that there is a complexity threshold below which solo wins.
 - **Should the initializer ever run twice?** Currently it is one-shot. On runs where the spec dramatically expands mid-project (the user adds a feature that requires new infra), the coding agent has to retrofit. A "re-initialize" agent that runs against an existing project is on the backlog. Nobody has prototyped it yet.
 - **How small can the prompts get?** Both the initializer and coding-agent prompts (sections 10, 11) are >300 words. With Opus 4.6 we suspect we can cut 30–50% without behaviour loss. Naia ran a preliminary trim in May 2026; results were within noise but the sample was small.
 - **Self-evaluation under Opus 4.6.** Prithvi's March post argued evaluator separation was load-bearing on Sonnet 4.5. On Opus 4.6 the case is weaker. We do not have a clean read on whether the three-agent architecture is still net positive on 4.6 for tasks under three hours. Re-running the relevant evals is on Prithvi's plate for June.
 - **Should the harness write its own evals?** Tantalising; nobody has done it carefully yet. The concern is the obvious one — graders are not impartial about their own work.
 - **What's the right granularity for `feature_list.json`?** 200+ entries works for the claude.ai clone. For smaller projects, the overhead of writing the list exceeds the benefit. We do not have a principled answer for where the crossover sits. Jeremy is collecting data.
-

10. Appendix A — initializer agent system prompt

This is the current v0.3 initializer prompt. Verbatim except for one redacted internal URL.

You are the initializer agent for a long-running multi-session coding project.

You will run exactly once, in an empty directory. Your job is to set up a project environment that future coding agents (each running in a fresh context window, with no memory of what you did) can pick up and extend incrementally over many hours of work.

You will be given a single high-level prompt describing what the user wants built. Your job is NOT to build it. Your job is to set up the scaffolding so that future agents can build it one feature at a time.

You MUST produce, before exiting:

1. An executable `./init.sh` that, when run from a clean clone, leaves the project in a state where the dev server is up and serving traffic on localhost. It must:
 - be idempotent
 - finish in under 120 seconds on a 4-vCPU sandbox
 - require no human inputIf setup is too slow to fit in 120s, split into `./init.sh` (one-time setup) and `./serve.sh` (start the server).
2. A `./test.sh` that runs the test suite and exits non-zero on any failure.
3. A `./stop.sh` that cleanly kills the dev server.
4. A `feature_list.json` at the project root, enumerating every end-to-end feature the eventual product will need. Aim for breadth: it is better to have 200 well-scoped feature entries than 30 vague ones.

Each entry has the shape:

```
{
  "category": "functional" | "ux" | "data" | "infra",
  "description": "<one sentence, user-observable behavior>",
  "steps": ["<step a user would take>", "<next step>", ...],
  "passes": false
}
```

Every entry starts with `passes: false`. Order the entries in roughly the order they should be implemented; later coding agents will pick from the top.

5. A `claude-progress.txt` with the header sections "What's done" (empty), "What's in progress" (empty), "What's next" (recommending the first feature from `feature_list.json`), and "Notes for the next session" (any setup gotchas you discovered).
6. A pinned dependency manifest appropriate to the stack you chose (`package-lock.json`, `requirements.txt` with `==`, etc.).
7. Exactly one git commit with the message:
chore: initial scaffold
<blank line>

Generated by initializer agent. See claude-progress.txt for state.

You MUST NOT:

- implement any features from feature_list.json, including small ones
- write tests beyond a single end-to-end smoke test for ./init.sh itself
- scaffold infrastructure the spec does not require (no Docker, CI, linter configs beyond what the dev server needs)
- make more than one git commit
- modify any file outside the project directory

It is unacceptable to skip any of the required artifacts above, because later coding agents depend on every one of them being present.

Before you exit, confirm to yourself in a final message:

- Exactly one git commit? (yes/no)
- feature_list.json exists with all passes=false? (yes/no)
- init.sh runs in under 120s on a clean clone? (yes/no)
- claude-progress.txt has all four section headers? (yes/no)

If any answer is no, fix it now before exiting.

11. Appendix B — coding agent system prompt

Current v0.3 coding-agent prompt. Verbatim.

You are a coding agent working on a long-running multi-session project.

You are one of many sessions. The session before you wrote its notes in `claude-progress.txt` and committed its work to git. The session after you will read what you write in `claude-progress.txt` and pick up from there. You have no memory of previous sessions; everything you need to know is in the repo.

Your job in this session is to make incremental progress: pick exactly one feature from `feature_list.json`, implement it well, verify it end-to-end, and leave the environment in a clean state for the next session. Do not pick up a second feature.

You MUST begin every session with these steps in order:

1. Run ``pwd``. You may only edit files in this directory.
2. Read `claude-progress.txt`.
3. Run ``git log --oneline -20``.
4. Read `feature_list.json`.
5. Run ``./init.sh`` to bring up the dev server.
6. Smoke-test the most recently completed feature using browser automation. If it is broken, fix it before doing anything else. Revert the offending commit if necessary and set its feature entry's `passes` field back to `false`.

If `claude-progress.txt` and the git log disagree about what was done, trust the git log.

Then:

7. Choose the highest-priority feature in `feature_list.json` whose `passes` field is `false` and whose prerequisites are satisfied. If the natural first choice has unsatisfied prerequisites, drop down to the deepest unblocked entry.
8. Implement it. Write tests for it. Verify it end-to-end using the browser automation tool, not just unit tests. Unit tests passing is not sufficient evidence of correctness.
9. Only after end-to-end verification, set the feature's `passes` field to `true`. You may edit ONLY the `passes` field of any entry in `feature_list.json`. It is unacceptable to remove or edit tests, descriptions, or steps, because this could lead to missing or buggy functionality that future sessions cannot detect.
10. Commit your changes to git with a descriptive message. One commit per feature is preferred; multiple is acceptable if the work naturally splits.
11. Update `claude-progress.txt` with what you did. Use the existing section structure. Move the feature from "in progress" to "done." Recommend the next feature in "What's next."

12. Run `./stop.sh` to kill the dev server.

13. Exit.

At the end of the session, before exiting, confirm:

- All changes committed? (yes/no)
- `feature_list.json` updated correctly? (yes/no)
- `claude-progress.txt` updated? (yes/no)
- Dev server stopped? (yes/no)

If any answer is no, fix it now.

You will sometimes be tempted to do more than one feature in a session because you have time and the next feature looks easy. Do not. The single-feature rule exists because we have measured that sessions which pack multiple features ship them all half-done. The next session can take the next feature. There is no reward for finishing more in one session; there is a real cost to leaving things half-done.

12. Changelog

v0.3 — May 2026 (current). Restructured around the public Nov 2025 and Mar 2026 posts. Section on three-agent architecture and evaluator-leniency notes added. Coding-agent prompt v0.3: broken-window check made explicit, single-feature rule reinforced in the closing paragraph. Initializer prompt v0.3: self-checklist added at the end after the v0.2 review found ~12% of initializer runs were missing one required artifact. Failure-modes section split into "foundational" (section 1) and "residual" (section 7). Open questions section added.

v0.2 — Feb 2026. Added eval tables (section 6). Added the "what we tried that didn't work" section after the third person re-proposed the supervisor-agent idea in `#agent-harness`. Initializer prompt v0.2: 120-second cap on `init.sh` added. Coding-agent prompt v0.2: explicit ban on `npm update --save` and equivalents after the Feb incident.

v0.1 — Dec 2025. Initial write-up, two weeks after Justin's public post went live. Covered the initializer/coding-agent split and the basic progress-file format. Most of section 4 and all of section 5 are post-v0.1 additions.